A Replicable Theory-Guided LLM Workflow for Measuring Legal Delegation and Administrative Discretion

Anonymous Author(s)
Anonymous Institution
Anonymous Location
anonymous@example.com

***Abstract*—Administrative discretion is a latent institutional construct defined in political economy as the residual policy latitude available to an agency, jointly determined by the authority delegated by a legislature and the constraints on its exercise. We translate this theory into a replicable, theory-guided large language model (LLM) workflow for coding U.S. financial-regulation laws. The workflow uses a binary delegation gate followed by separate evaluations of delegated authority (low/high) and statutory constraints (strong/weak) to derive a five-category ordinal discretion rank. We implement this measurement instrument in Delegation Workflow Studio[1], a research prototype that exposes prompts, settings, outputs, and validation rules as a versioned, auditable workflow. We evaluate the Delegation Gate on the full delegation benchmark and compare three rank-classification strategies across multiple language models on the subset with complete expert-coded discretion ranks. By separating generative judgments from deterministic theoretical mappings and preserving the execution record, the system enforces specified consistency rules and supports reproducible auditing and rerunning.**

***Index Terms*—legal natural language processing, large language models, legislative delegation, administrative discretion, reproducible workflows, quantitative legal coding**

I. Introduction

Researchers studying political institutions routinely convert complex legal texts into structured measures of authority, constraint, and institutional design [1]. A key example is administrative discretion, which represents the residual policy latitude available to an executive actor after jointly considering the amount of authority delegated by a legislature and the legal-administrative constraints governing its use [2, 3].

Although one-shot LLM classification provides a simple baseline, it can obscure the intermediate judgments underlying a final rank and can produce outputs that are inconsistent with the measurement model. Single-prompt approaches are also difficult to audit because the relationships among delegation, authority, constraints, and the final classification remain embedded in one generative output.

We address this problem by representing the underlying political-economy theory as an executable workflow graph. Delegation is modeled as a logically prior gate. If delegation is present, the workflow separately evaluates the amount of delegated authority and the strength of constraints on its exercise, combining these values to derive the final discretion rank. We implement the approach in Delegation Workflow Studio, a research prototype that represents and executes prompts, settings, intermediate outputs, and validation rules as a versioned and auditable workflow graph.

The principal contribution is replicability. The theory-guided design makes the workflow substantively meaningful by ensuring that its stages and dependencies align with the established measurement model rather than with an arbitrary sequence of prompts.

This paper makes four principal contributions:

1) The paper develops a replicable LLM-based measurement instrument for legal delegation and administrative discretion. The complete workflow, rather than an isolated prompt, is archived, versioned, and auditable.
2) The workflow is theory-guided: the established measurement model determines the variables, stage ordering, conditional dependencies, and permitted relationships between the component classifications and the final rank.
3) The system separates generative LLM judgments from deterministic theoretical mappings and preserves the intermediate evidence needed to diagnose disagreements.
4) The prototype enables domain researchers to apply and inspect the measurement process without directly managing prompts, APIs, model orchestration, or structured-output parsing.

II. Theory and Operationalization of Administrative Discretion

A. Theoretical framework

Delegation is the transfer of policymaking authority from a legislature to an executive or administrative actor, allowing the agent to make choices that alter policy or regulate conduct. However, delegation alone does not determine administrative discretion. Legislatures routinely impose substantive and procedural constraints—such as administrative standards, approval requirements, deadlines, reporting obligations, and oversight reviews—to restrict the agent's latitude.

We define administrative discretion as the residual policy latitude available to an agency after jointly considering the

[1]An interactive version of the Delegation Workflow Studio dashboard is anonymized for double-blind review.

delegation of authority and the constraints governing its use, formalized as:

$$\text{Discretion} = D(1 - C),$$

where D represents delegated authority and C is the constraint index [2, 3].

B. Ordinal operationalization

The LLM workflow translates this continuous theory into a five-category ordinal coding. Rank 0 is determined by the absence of a qualifying delegation. Conditional on delegation, the remaining classes represent the four cells produced by crossing the authority classification (low or high) with the constraint classification (strong or weak). The final rank is therefore derived from these component judgments as illustrated in Figure 1.

C. Theory-guided computational inference

We use the term *theory-guided AI* to describe a computational design in which substantive theory structures the inference process. Delegation is logically prior to discretion. Conditional on a qualifying delegation, the workflow evaluates authority and constraints as separate components and then uses their relationship to interpret or validate the final discretion rank:

$$\text{Delegation} \rightarrow \{\text{Authority}, \text{Constraints}\} \rightarrow \text{Discretion rank}.$$

Figure 1 summarizes the theory-to-classification mapping used in the main workflow.

D. Problem Formulation

Let x_i denote the legal text for law i. The theoretical measurement model contains three component variables. Let

$$d_i \in \{0, 1\}$$

indicate whether the law delegates qualifying policymaking authority,

$$a_i \in \{\text{Low}, \text{High}\}$$

denote the amount of delegated authority, and

$$c_i \in \{\text{Strong}, \text{Weak}\}$$

denote the strength of statutory constraints.

The substantive theory defines a deterministic mapping from these components to the discretion rank:

$$y_i = m(d_i, a_i, c_i),$$

where

$$m(d_i, a_i, c_i) = \begin{cases} 0, & d_i = 0, \\ 1, & d_i = 1,\ a_i = \text{Low},\ c_i = \text{Strong}, \\ 2, & d_i = 1,\ a_i = \text{Low},\ c_i = \text{Weak}, \\ 3, & d_i = 1,\ a_i = \text{High},\ c_i = \text{Strong}, \\ 4, & d_i = 1,\ a_i = \text{High},\ c_i = \text{Weak}. \end{cases}$$

This mapping defines the substantive meaning of the five discretion ranks. It is part of the measurement model rather than a learned relationship.

The workflow uses versioned LLM stages to estimate the component variables:

$$\hat{d}_i = g_D(x_i),$$

$$\hat{a}_i = g_A(x_i, \hat{d}_i),$$

$$\hat{c}_i = g_C(x_i, \hat{d}_i, \hat{a}_i).$$

The component stages also return structured evidence and rationales, denoted collectively by r_i. Their prompts, inputs, outputs, evidence, and model settings are retained in the execution trace.

The system then applies one of three rank-classification strategies. Let

$$\mathcal{S} = \{\text{Cascade}, \text{Multiclass}, \text{TwoBand}\}$$

denote the Sequential Ordinal Cascade, Direct Multiclass Rank Prediction, and Two-Band Rank Classification strategies, respectively. For strategy $s \in \mathcal{S}$,

$$\tilde{y}_i^{(s)} = h_s(x_i, \hat{d}_i, \hat{a}_i, \hat{c}_i, r_i).$$

The Sequential Ordinal Cascade exploits the outcome's ordered structure. The delegation gate first separates Rank 0 from Ranks 1–4. Conditional on delegation, the cascade proceeds in stages, with the first separating Rank 1 from Ranks 2–4, the second separating Rank 2 from Ranks 3–4, and the final stage separating Rank 3 from Rank 4. Each positive decision produces an early exit, while each negative decision routes the case to the next stage.

The Direct Multiclass strategy predicts one of Ranks 1–4 in a single ranking step using the stored component outputs and rationales. The Two-Band strategy first separates the lower-rank band $\{1, 2\}$ from the higher-rank band $\{3, 4\}$, then classifies within the selected band.

The theoretical mapping and the ranking strategies, therefore, perform different functions. The mapping $m(d_i, a_i, c_i)$ defines the construct, whereas the functions $h_s(\cdot)$ estimate the final rank under alternative computational strategies.

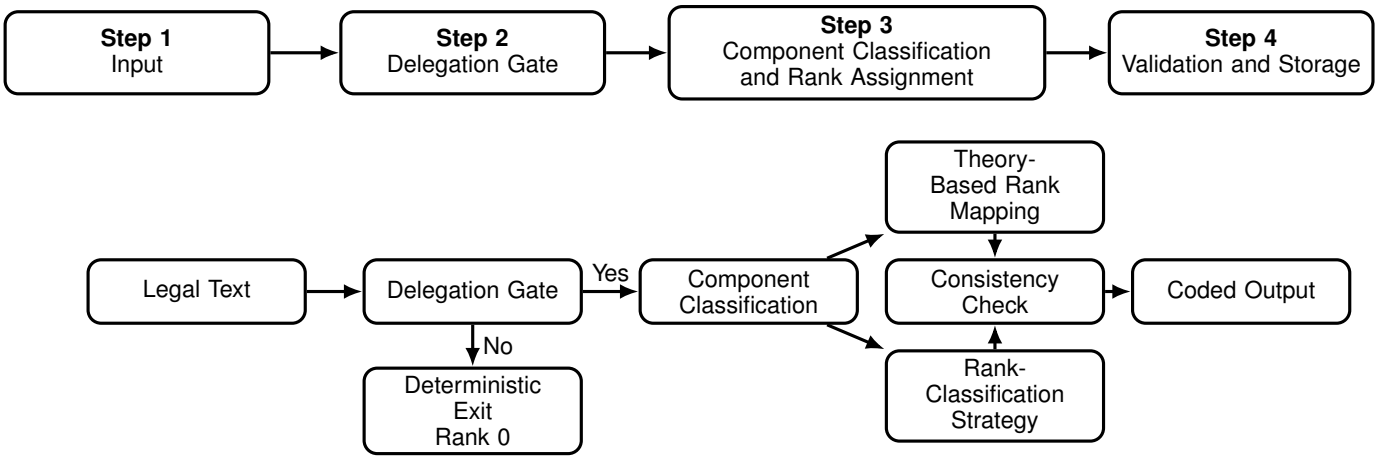
For each law, the system also computes the theory-implied rank; that is, the rank is determined by the estimated component classifications,

$$y_i^{\text{map}} = m(\hat{d}_i, \hat{a}_i, \hat{c}_i)$$

and compares it with the strategy-specific prediction. Define the consistency indicator

$$q_i^{(s)} = \mathbf{1}\left[\tilde{y}_i^{(s)} = y_i^{\text{map}}\right].$$

A value of $q_i^{(s)} = 1$ indicates that the rank predicted by strategy s agrees with the rank implied by the model's own delegation, authority, and constraint classifications. A disagreement is retained and flagged for review rather than silently overwritten. This design allows the evaluation to distinguish benchmark agreement from internal theoretical consistency.



Variables and records by step

Step 1. Input: A legislative summary or statutory text, together with a stable law identifier.

Step 2. Delegation Gate: $\hat{d}_i \in \{0, 1\}$, indicating whether the law contains a qualifying delegation.

Step 3. Component Classification and Rank Assignment: $\hat{a}_i \in \{\text{Low}, \text{High}\}$, $\hat{c}_i \in \{\text{Strong}, \text{Weak}\}$, and r_i, containing supporting evidence and rationale. The workflow records both $y_i^{\text{map}} = m(\hat{d}_i, \hat{a}_i, \hat{c}_i)$, the rank implied by the component classifications, and $\tilde{y}_i^{(s)}$, the rank predicted by strategy s.

Step 4. Validation and Storage: $q_i^{(s)} = \mathbf{1}[\tilde{y}_i^{(s)} = y_i^{\text{map}}]$, together with the final rank, component values, evidence, model settings, and execution trace.

Fig. 1: Four-stage theory-guided classification workflow. Step 1 ingests a legislative summary or statutory text and a stable law identifier. Step 2 determines whether the law contains a qualifying delegation and assigns Rank 0 when delegation is absent. Step 3 classifies authority and constraints and produces both the theory-based and strategy-predicted ranks. Step 4 compares the two rank outputs and stores the coded variables, evidence, settings, and execution trace.

III. Related Work

The present study extends a cumulative research program on legislative delegation and administrative discretion. Early formal work conceptualized discretion as the range of policy choices left to an administrative actor and examined how legislatures balance administrative expertise against the risks of bureaucratic drift [4]. Later empirical work operationalized these concepts using a database of U.S. financial-regulation laws, combining measures of delegated authority and legal constraints into a discretion index [3, 5, 6]. Computational studies have since demonstrated the utility of algorithmic approaches for recovering discretion classifications from legal text [7, 8, 9]. Rather than using theory-derived variables only as inputs to a flat classifier, we use the theory to structure the LLM workflow itself.

LLMs have shown promise as low-cost annotators across various text-annotation and qualitative tasks [10, 11]. However, prompted generative models often perform poorly when applying domain-expert codebooks to specialized historical or legal texts [12]. This contrast motivates our conservative design: LLM coding must be evaluated against a controlled benchmark and remain subject to human review. Legal NLP benchmarks such as LexGLUE and LegalBench also emphasize task-specific evaluation and expert-designed categories [13, 14].

Our work also builds on weak supervision and interactive human feedback systems [15, 16], alongside visual prompt-chaining systems that decompose LLM tasks into modular, controllable steps [17, 18]. Rather than claiming to introduce human-in-the-loop annotation or prompt chaining, we integrate these computational paradigms with a formal political-economy measurement model of legislative delegation.

IV. Theory-Guided Workflow Architecture

The workflow order is grounded in the substantive theory. It is not merely an engineering strategy to reduce prompt length. A law cannot confer administrative discretion without first transferring authority. Once delegation is established, residual discretion depends jointly on the amount of authority transferred and the constraints on its use. The computational architecture therefore mirrors the conceptual structure of the measurement model.

The system was developed iteratively to align LLM judgments with the expert codebook rules. The crucial methodological shift was from treating a single prompt as the coding instrument to treating the structured, multi-stage workflow graph as the coding instrument.

A. From theory to ordered computational judgments

The final rank depends on earlier judgments. A model should first determine whether a qualifying delegation exists, then inventory administrative actors and authority, assess the

breadth and centrality of that authority, identify constraints, and only then estimate residual leeway. If no delegation exists, the workflow assigns rank 0 deterministically instead of spending another model call on an inapplicable question. If delegation exists, the rank stage receives the earlier values and rationales explicitly.

This dependency structure addresses a recurrent failure of independent-column coding: a later variable such as `discretion_rank` cannot reliably use an earlier delegation decision unless the pipeline stores and forwards it. In the prototype, execution order is computed from the graph rather than assumed from the visual order of columns. Each node records inputs, outputs, status, timing, and, where applicable, evidence and rationale.

B. Workflow primitives

The implemented engine supports several node types:

1) **Document Input nodes** ingest the source text and attach stable metadata (such as public law numbers and publication years) to define a unique document instance.
2) **LLM nodes** send natural language prompts and source texts to the selected model provider, parsing the structured JSON output into typed fields (booleans, strings, enums, lists).
3) **Condition nodes** evaluate boolean expressions (e.g., matching the output of a prior delegation gate) and route the execution path to the appropriate successor branch.
4) **Set Value nodes** deterministically assign default values to output fields without triggering LLM calls, bypassing downstream nodes when prior conditions dictate a fixed result.
5) **Validation nodes** evaluate user-defined rules and integrity constraints (e.g., checking that discretion rank is zero if delegation is false) and record validation flags or raise validation errors when outputs are anomalous.
6) **Output nodes** group and serialize the final variables and rationales to expose them on the dashboard.

Table I details the step-by-step algorithmic execution sequence mapping these primitives to the discretion-coding workflow.

The workflow retains both the strategy-specific prediction $\tilde{y}_i^{(s)}$ and the theory-implied rank y_i^{map}. Strategy-specific predictions are used for comparative evaluation and are not silently replaced when they conflict with the deterministic component mapping. Instead, the validation node records the disagreement with an inconsistency flag. This separation allows researchers to evaluate both agreement with the expert-coded benchmark and consistency with the rules used to construct the discretion rank.

A one-shot prediction collapses the relationships among delegation, authority, constraints, and discretion into an opaque judgment. The workflow makes the measurement assumptions inspectable and testable because researchers can locate a disagreement at the Delegation Gate, authority classification, constraint classification, rank-classification strategy, or consistency check. Figure 2 illustrates the reproducibility architecture, showing how source inputs, versioned instruments, execution configurations, and trace logs are stored to ensure auditability.

TABLE I: Algorithmic execution of the theory-guided discretion workflow.

Step	Operation and Logic
1	**Load Source Text:** Read the legislative summary or statutory text x_i and attach stable identifiers.
2	**Delegation Gate (LLM):** Estimate $\hat{d}_i = g_D(x_i) \in \{0, 1\}$ and retain the supporting evidence and rationale.
3	**Early Exit (Deterministic):** If $\hat{d}_i = 0$, set the theory-implied rank $y_i^{\text{map}} = 0$, assign the required default values, and stop the delegated-discretion branch.
4	**Component Classification (LLM):** Conditional on $\hat{d}_i = 1$, identify the relevant administrative actors, classify delegated authority $\hat{a}_i \in \{\text{Low}, \text{High}\}$, classify statutory constraints $\hat{c}_i \in \{\text{Strong}, \text{Weak}\}$, and retain the supporting evidence and rationales r_i.
5	**Strategy-Specific Rank Prediction (LLM):** Apply the selected rank-classification strategy. The Sequential Ordinal Cascade uses successive early-exit binary decisions; Direct Multiclass predicts Ranks 1–4 in one step; and Two-Band first predicts a lower/higher rank band and then classifies within the selected band.
6	**Theoretical Consistency Validation (Deterministic):** Compute $y_i^{\text{map}} = m(\hat{d}_i, \hat{a}_i, \hat{c}_i)$ and compare it with the strategy-specific prediction $\tilde{y}_i^{(s)}$. Record the consistency indicator $q_i^{(s)}$ and flag any disagreement.
7	**Output Serialization:** Store the component classifications, theory-implied rank, strategy-specific rank, consistency flag, evidence, prompts, model settings, execution trace, runtimes, token use, cost, and final reported output.

C. System Implementation Status

To distinguish between the features fully operational in the prototype, those partially implemented, and those planned for future iterations, Table II summarizes the current capability matrix of the visual builder and execution studio.

TABLE II: System implementation and capability matrix.

Capability	Implemented	Evaluated	Planned
Versioned workflow definitions	Yes	Yes	
Conditional branching	Yes	Yes	
Deterministic rank mapping	Yes	Yes	
Prompt and output logging	Yes	Yes	
Workflow hash (SHA-256)	Yes	Yes	
Case-level manual override	Yes	Partial	
Version-comparison interface	Partial	Partial	
Formal approval workflow			Yes

V. Human Validation and Research Governance

Human expertise enters before, during, and after model execution. Before execution, domain experts define the construct, identify its constituent dimensions, delimit the source universe, specify coding rules, and determine the relationships

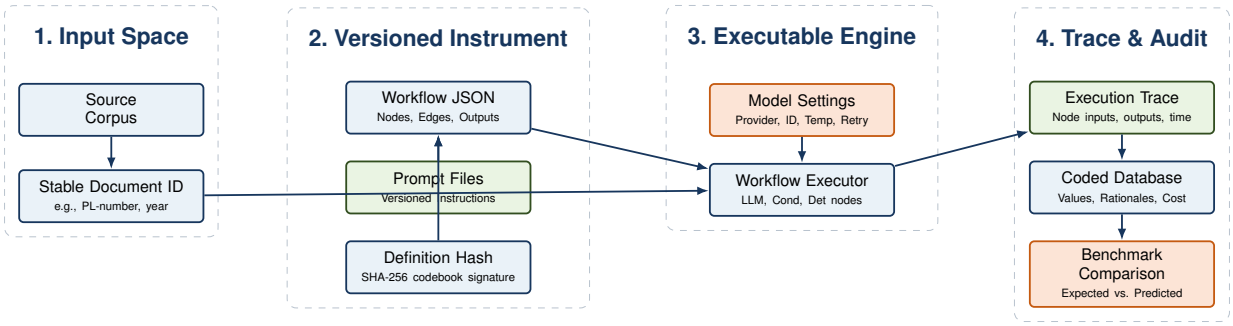


Fig. 2: Reproducibility and execution tracking pipeline. The system archives the source corpus, the workflow JSON schema and definition hash (SHA-256), model configurations, and a comprehensive node-level execution trace to ensure that any coded dataset can be audited and reconstructed.

that the model must preserve. During evaluation, experts distinguish extraction errors from conceptual boundary cases. After evaluation, they decide whether a disagreement warrants a local case correction, a revision of the operational rule, or reconsideration of the historical benchmark.

Human-in-the-loop design is the mechanism by which the theoretically structured measurement instrument is governed and validated. Case corrections are local and do not silently alter subsequent classifications; general codebook adjustments are promoted explicitly through versioned workflow releases.

A. Method-level revision

At the method level, researchers revise the codebook, source policy, stage prompts, dependencies, output definitions, and deterministic mappings. A method-level revision changes the general measurement rule and should therefore create a new version and controlled rerun before adoption. This protects the comparability of coded observations across documents and model runs.

B. Case-level adjudication

For an individual coded law, a researcher can inspect final values and node-by-node execution traces, manually override a selected value and rationale, or request AI re-evaluation with corrective feedback. History distinguishes initial AI output, manual override, and AI re-evaluation. A manual correction is local to that law; it does not automatically change the codebook, the prompt, or other cases.

This separation is scientifically useful. Case correction protects the dataset from a known error, whereas method revision changes the coding rule. The prototype therefore avoids an opaque "learning" process in which corrections silently alter subsequent classifications.

VI. Benchmark and Experimental Design

A. Benchmark and source discipline

The benchmark consists of U.S. financial-regulation laws previously published and adjudicated under a consistent coding protocol. The evaluation corpus contains $N = 169$ statutory summaries covering the period 1950–2013, consisting of 161 delegation-positive and 8 delegation-negative laws. We treat these observations as the operational benchmark for replicating measurements, not as infallible legal truth. The source universe is methodologically important: the benchmark labels were produced from CQ summaries, so benchmark runs use the same summary-level information set to maintain consistency.

B. Evaluation levels

The empirical evaluation asks whether LLMs can reproduce a theory-based measurement instrument, not merely whether they can generate plausible descriptions of statutes. The evaluation therefore occurs at several levels: whether the model identifies qualifying delegation, correctly characterizes the amount of authority, correctly characterizes the strength of constraints, produces a final rank consistent with those component judgments, and agrees with the historical benchmark.

We separate the evaluation into two distinct prediction tasks:

1) **Delegation Stage**: Binary delegation classification evaluated on the full available delegation benchmark ($N = 169$).
2) **Residual Discretion Stage**: Ordinal rank classification (1–4) evaluated only when a qualifying delegation exists ($N = 161$).

C. Controlled prompt and workflow comparison

To evaluate discretion ranking, we implement and compare three staged computational strategies against a one-shot baseline. The strategies are summarized in Table III.

D. Experimental protocol

The experimental protocol is specified as follows:

- **Models**: We evaluate Gemini 3.5 (`gemini-3.5-flash`), Qwen 3 (`qwen/qwen3-235b-a22b-2507`), DeepSeek (`deepseek-v4-flash`), Kimi K2.5 (`moonshotai/kimi-k2.5`), and GPT-OSS (`openai/gpt-oss-120b`).
- **Settings**: The system uses a temperature of 0.0 to reduce sampling variation. Because commercial APIs may still exhibit nondeterminism or provider-side changes, output stability is evaluated through repeated runs and the exact model identifiers and execution timestamps are retained.
- **Output Format**: Nodes output structured JSON adhering to strict Pydantic schemas.

TABLE III: Comparison of evaluation strategies.

Strategy	Inputs	Decision process	Output
One-shot baseline	Law text	Single LLM prediction with no intermediate guidance	Discretion Rank 0–4
Sequential Ordinal Cascade	Stage outputs	Evaluates early-exit binary splits: Rank 0 vs. Ranks 1–4 (delegation gate), then Rank 1 vs. Ranks 2–4, then Rank 2 vs. Ranks 3–4, and Rank 3 vs. Rank 4	Discretion Rank 0–4
Direct Multiclass Rank Prediction	Stage outputs	Predicts 1–4 directly using extracted authority and constraints	Discretion Rank 0–4
Two-Band Rank Classification	Stage outputs	Low-rank vs. high-rank band split, then within-band classification	Discretion Rank 0–4

- **Retry & Failures**: If a model call fails, returns an invalid structured output, or times out, the system retries the call up to three times using the identical prompt, parameters, and schema settings. Persistent failures are recorded in the execution log and excluded or reported according to the stated evaluation protocol.

E. Metrics

For the binary delegation task, we report accuracy, balanced accuracy, class-specific precision and recall, macro-F1, and the confusion matrix. For the ordinal discretion task, we report exact-match accuracy, within-one accuracy, mean absolute error, linear weighted kappa, quadratic weighted kappa, and the ordinal confusion matrix.

VII. Experimental Findings

A. Binary delegation

Table IV reports binary delegation screening results on the full available delegation benchmark ($N = 169$). The benchmark is severely imbalanced (161 positive cases and only 8 negative cases).

TABLE IV: Binary delegation stage screening results ($N = 169$).

Metric	Looser Screening	Delegation Gate
Overall Accuracy	94.67%	91.72%
Balanced Accuracy	73.45%	89.71%
Positive Recall (Delegation)	96.89%	91.93%
Negative Recall	50.00%	87.50%
Positive Precision	97.50%	99.33%
Negative Precision	44.44%	35.00%
Macro-F1	72.13%	72.74%
TP / FN / TN / FP	156 / 5 / 4 / 4	148 / 13 / 7 / 1

While the Looser Screening baseline yields slightly higher raw accuracy due to the extreme class imbalance, the conservative Delegation Gate detects seven of the eight rare negative cases, producing a substantially higher balanced accuracy and macro-F1.

B. Diagnosing disagreement

The law PL83-577 illustrates why source discipline and intermediate evidence matter. The benchmark codes it as non-delegation (rank 0). An exploratory staged run instead identified Securities and Exchange Commission rulemaking and exemption authority and returned delegation. This disagreement could reflect an overbroad coding rule, a source-universe difference, or a mismatch between historical summary coding and full statutory detail. The trace does not by itself determine which label is correct; it makes the source of the disagreement visible for expert adjudication.

C. Discretion rank and model comparison

Table V reports the cross-model evaluation of the three ranking strategies (Sequential Ordinal Cascade, Direct Multiclass Rank Prediction, and Two-Band Rank Classification) alongside an unconstrained One-Shot Rank Prediction baseline on the expert-coded discretion benchmark ($N = 169$).

As shown in Table V, both the Sequential Ordinal Cascade and Direct Multiclass strategies consistently outperform the unconstrained One-Shot Baseline across models. On Gemini 3.5, the Direct Multiclass strategy increases exact-match accuracy from 28.6% to 41.9% and Within-1 accuracy from 62.1% to 76.9%, while reducing Mean Absolute Error from 1.250 to 0.915.

The ordinal error distribution reveals that classification errors are overwhelmingly adjacent (± 1 rank). Across all evaluated models, off-by-one predictions account for over 85% of total misclassifications (e.g., classifying a Rank 2 law as Rank 1 or Rank 3), whereas multi-rank errors (≥ 2 categories) are rare ($< 15\%$). Consequently, Within-1 accuracy reaches 76.9%, while linear and quadratic weighted kappa (κ_w) remain low (0.01 to 0.20). This pattern highlights fine-grained boundary ambiguity in historical five-class legal discretion coding rather than arbitrary prediction, underscoring why the system flags inconsistency for expert review rather than relying on unvalidated generative output.

TABLE V: Cross-model evaluation results across ranking strategies and unconstrained baseline ($N = 169$).

Model	Strategy	Exact	W1	MAE	Linear κ_w	Quad κ_w
Gemini 3.5	One-Shot Baseline (Unconstrained)	28.6%	62.1%	1.250	0.051	0.042
	Sequential Ordinal Cascade	29.9%	75.2%	1.077	0.104	0.094
	Direct Multiclass Prediction	41.9%	76.9%	0.915	0.207	0.157
	Two-Band Classification	29.9%	65.8%	1.188	0.044	0.010
Qwen 3	Sequential Ordinal Cascade	25.6%	65.9%	1.225	0.062	0.059
	Direct Multiclass Prediction	32.6%	64.3%	1.171	0.105	0.093
	Two-Band Classification	17.8%	48.8%	1.620	0.030	0.010
DeepSeek	Sequential Ordinal Cascade	20.9%	57.4%	1.434	0.022	0.010
	Direct Multiclass Prediction	29.5%	71.3%	1.140	0.055	0.014
	Two-Band Classification	8.5%	41.9%	1.837	-0.014	-0.020
Kimi K2.5	Sequential Ordinal Cascade	30.0%	63.3%	1.317	0.133	0.094
	Direct Multiclass Prediction	21.7%	60.0%	1.433	0.076	0.067
	Two-Band Classification	15.0%	45.0%	1.700	-0.008	-0.003
GPT-OSS	Sequential Ordinal Cascade	20.0%	55.3%	1.482	0.056	0.067
	Direct Multiclass Prediction	32.9%	69.4%	1.129	0.106	0.072
	Two-Band Classification	9.4%	37.6%	1.871	0.007	0.021

D. Limitations

While the proposed framework improves replication and logical consistency, three main limitations remain. First, the evaluation relies on legislative summaries rather than full statutory text. Summaries have a more uniform structure, and applying the prompts directly to full statutory text or omnibus acts may introduce noise due to parsing complexity and document length limits. Second, the current prompts and expert benchmarks are tailored to the financial-regulation domain (banking and securities). The external validity of these prompts for other administrative policy domains, such as environmental or labor law, remains to be verified. Third, although deterministic validation enforces specified consistency rules, the underlying LLM classifications remain sensitive to model choice, prompt wording, nondeterministic API behavior, and provider-side model updates.

E. Novelty claim

The paper relies on well-established theories of delegation or administrative discretion. Our contribution is a replicable LLM measurement architecture that implements a theoretical model through explicit variables, ordered stages, deterministic consistency rules, and preserved execution records.

VIII. Replication Workflow

The current prototype can be replicated through the following research-facing sequence:

1) **Prepare the corpus.** Upload CQ summaries or corresponding major-provisions text and retain stable law identifiers. Keep full statutes in a separately labeled exploratory collection.
2) **Create the instrument.** Duplicate the delegation-and-discretion workflow template. Review every stage's theoretical purpose, prompt, required inputs, typed outputs, and final/internal visibility.
3) **Validate and test.** Run representative positive, negative, and boundary cases. Inspect graph validation, final outputs, and node-level traces.
4) **Freeze a version.** Before an evaluation run, the workflow definition, prompts, model settings, and source set are frozen and assigned a version identifier. Subsequent edits create a new workflow version and do not alter the record of prior runs.
5) **Run a controlled evaluation.** Apply the same workflow and source set across selected models or strategies. Record model identifiers, settings, timestamps, failures, token use, and cost.
6) **Compare with the benchmark.** Map benchmark columns, calculate binary class-specific metrics and ordinal rank metrics, and inspect every mismatch.
7) **Intervene explicitly.** Correct a case locally when appropriate. If a disagreement reveals a general coding-rule problem, revise the workflow as a new version, rerun the development set, and evaluate the revised version on data not used to make that change when such data are available.
8) **Export the dataset.** Preserve final variables alongside the workflow version, model, source universe, and audit history needed for downstream quantitative analysis.

IX. Future Work

Future work will evaluate the workflow on a held-out set of laws, measure component-level authority and constraint performance, and test external validity in an additional policy domain. Further system work will strengthen workflow-version comparison, approval controls, and case-adjudication interfaces. Independent expert recoding would also permit direct estimation of intercoder reliability and clarify the upper bound on model-benchmark agreement.

X. Conclusion

This paper began with a longstanding measurement problem: how to recover delegated authority and administrative discretion from legal text. Political-economy theory provides a clear conceptual answer. Legislatures may transfer varying amounts of authority to administrative actors and impose

varying constraints on their exercise; discretion is the residual policy latitude of those joint choices.

The methodological challenge is to make that theory executable at scale without reducing it to either a black-box classifier or a single opaque prompt. We address this challenge with a theory-guided LLM workflow. Delegation, authority, and constraints are treated as distinct yet interdependent components, and the final category is derived from their theoretically specified relationship. Human experts govern the source universe, codebook, validation process, case adjudication, and instrument revision.

The resulting system is a replicable measurement instrument whose computational structure is guided by an established theory of delegation and discretion. Replicability is the primary contribution; theory-guided design explains why the workflow's stages, dependencies, and validation rules are substantively meaningful. The system does not remove judgment from institutional measurement; instead, it makes judgment explicit, structured, inspectable, and reproducible.

References

[1] J. Grimmer and B. M. Stewart, "Text as data: The promise and pitfalls of automatic content analysis methods for political texts," *Political Analysis*, vol. 21, no. 3, pp. 267–297, 2013.

[2] D. Epstein and S. O'Halloran, *Delegating Powers: A Transaction Cost Politics Approach to Policy Making under Separate Powers*. Cambridge: Cambridge University Press, 1999.

[3] S. O'Halloran *et al.*, "Big data and the regulation of banking and financial services," *Banking & Financial Services Policy Report*, vol. 34, no. 12, pp. 1–10, 2015.

[4] D. Epstein and S. O'Halloran, "Administrative procedures, information, and agency discretion," *American Journal of Political Science*, vol. 38, no. 3, pp. 697–722, 1994.

[5] S. O'Halloran *et al.*, "Data science and political economy: Application to financial regulatory structure," *RSF: The Russell Sage Foundation Journal of the Social Sciences*, vol. 2, no. 7, 2016.

[6] T. Groll, S. O'Halloran, and G. McAllister, "Delegation and the regulation of u.s. financial markets," *European Journal of Political Economy*, vol. 70, p. 102058, 2021.

[7] S. O'Halloran *et al.*, "Big data and the regulation of financial markets," in *Proceedings of the 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. ACM, 2015, pp. 1118–1124.

[8] ——, "Computational data sciences and the regulation of banking and financial services," in *Social Network Analysis Lecture Notes*, R. Alhajj, Ed. New York: Springer, 2016.

[9] ——, "Big data and graph theoretic models: Simulating the impact of collateralization on financial system," in *Proceedings of the 2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. ACM, 2017, pp. 1056–1064.

[10] F. Gilardi, M. Alizadeh, and M. Kubli, "Chatgpt outperforms crowd workers for text-annotation tasks," *Proceedings of the National Academy of Sciences*, vol. 120, no. 30, p. e2305016120, 2023.

[11] S.-C. Dai, A. Xiong, and L.-W. Ku, "LLM-in-the-loop: Leveraging large language model for thematic analysis," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 9993–10 001.

[12] R. Thalken, E. Stiglitz, D. Mimno, and M. Wilkens, "Modeling legal reasoning: LM annotation at the edge of human agreement," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 9252–9265.

[13] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. Katz, and N. Aletras, "LexGLUE: A benchmark dataset for legal language understanding in english," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2022, pp. 4310–4330.

[14] N. Guha, J. Nyarko, D. E. Ho, C. Ré, A. Chilton *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, arXiv:2308.11462.

[15] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, "Snorkel: Rapid training data creation with weak supervision," *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.

[16] G. V. Cormack and M. R. Grossman, "Evaluation of machine-learning protocols for technology-assisted review in electronic discovery," in *Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2014, pp. 153–162.

[17] T. Wu, M. Terry, and C. J. Cai, "Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, 2022.

[18] T. Wu, E. Jiang, A. Donsbach, J. Gray, A. Molina, M. Terry, and C. J. Cai, "Promptchainer: Chaining large language model prompts through visual programming," in *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. ACM, 2022.